EXACT Pipeline: Solution Description

1. Datasets Used

1.1. Official EXACT Dataset

- **Name**: EXACT Official Benchmark Dataset.
- **Source/Origin**: Official EXACT competition dataset.
- **Number of Samples Used**: 100% of the provided training and validation samples.
- **Sample Entries**:
  - *Physics*: "A car travels 100 meters in 5 seconds. What is its average speed?"
  - *Logic*: "If all cats are mammals, and Tom is a cat, is Tom a mammal?"

1.2. External Knowledge Corpus (Graph & Vector DB)

- **Name**: Neuro-Symbolic Physics & Logic Rules Corpus.
- **Source/Origin**: Synthetic data and rules extracted from standard open-source Physics and Math libraries, formatted to populate our vector database (`ChromaDB`) and knowledge graph (`NetworkX`).
- **Number of Samples Used**: ~1,500 structured nodes (formulas and logic rules).
- **Sample Entries**:
  - *Physics Formula*: `{"concept": "Kinematics", "formula": "v = d / t", "variables": {"v": "velocity", "d": "distance", "t": "time"}}`
  - *Logic Rule*: `{"rule": "Modus Ponens", "expression": "Implies(And(P, Implies(P, Q)), Q)"}`

---

2. Approach and Method

Our pipeline adopts a **Neuro-Symbolic** architecture, explicitly designed to eliminate LLM hallucinations, ensure deterministic correctness, and maintain dynamic resource efficiency. The workflow is divided into 5 chronological phases:

Phase 0: Offline Knowledge Compilation

Before handling live queries, the system pre-processes external knowledge. Natural Language rules, physics formulas, and logic facts are translated by an offline LLM into computable assets (First-Order Logic (FOL) expressions, Z3 scripts, and SymPy equations). These are mapped and ingested into a Neuro-Symbolic **HybridDB** (a tightly coupled architecture combining `ChromaDB` for vectors and `NetworkX` for knowledge graphs).

Phase 1: Semantic Extraction & Adaptive Routing

When a query arrives, it undergoes keyword-based heuristics to determine its global domain (Physics or Logic). Instead of sending the raw prompt to a massive LLM, we utilize `Gemma-3-1B-it` as a lightweight Semantic Extractor.

- It efficiently parses the query to extract the intent, conditions, targets, and Named Entities (NER).

- Simultaneously, syntactic parsing evaluates the query's complexity (κ). An Exponential Moving Average (EMA) utility function evaluates this complexity alongside real-time server load to route the query adaptively.

Phase 2: Neuro-Symbolic Hybrid Retrieval

To provide the reasoning engine with perfect context, we implement a multi-stage retrieval process:

1. **Dense Retrieval:** The extracted entities are embedded via `BAAI/bge-small-en-v1.5` to fetch the Top-50 closest nodes from `ChromaDB`.
2. **Cross-Encoder Reranking:** To filter out vector-space noise, `BAAI/bge-reranker-v2-m3` scores the actual semantic relevance, narrowing it down to the Top-3 nodes.
3. **Subgraph Extraction:** Using Breadth-First Search (BFS) and Steiner Tree approximations on the GraphDB, we extract the neighboring nodes of the Top-3 results. The final context relevance is calculated using a hybrid scoring equation combining Vector Score and Graph Centrality:

$$Score_{total} = \alpha \times VectorScore + (1 - \alpha) \times PageRankScore$$

Phase 3: Domain-Specific Reasoning Pipelines

Based on the extracted intent and domain, the pipeline bifurcates to maximize efficiency:

- **The Physics Pipeline:** The system attempts to map the subgraph context to an exact formula. If a direct match is found (Fast Path), it bypasses the LLM entirely and solves it using the Fast `SymPy` Solver. If the query is complex, it utilizes Jinja2 Constraint Prompting to force the Main LLM (`Qwen2.5-7B-Instruct`) to generate a Python script using `SymPy` to solve the physics equations programmatically.
- **The Logic Pipeline:** The Intent Router dictates the solver. For verification or truth-finding questions, the context is converted into FOL premises and sent to the `z3` Symbolic Solver (using Proof by Contradiction and SAT Solving). For path-finding logic, the Main LLM generates `NetworkX` code to traverse logic trees. A standard LLM fallback is reserved strictly for open-ended analysis.

Phase 4: Deterministic Execution & Sandbox

To prevent arbitrary code execution and ensure absolute safety, all Python code generated by the Main LLM is intercepted and executed in a highly restricted Secure Sandbox.

- **AST Inspection:** The code is parsed into an Abstract Syntax Tree (`ast.parse`). Any malicious nodes (e.g., `import os`, `sys`, `eval()`, `exec()`) are immediately blocked to prevent RCE attacks.
- **Isolated Subprocesses:** Approved code runs in an isolated child process with strict timeouts (e.g., 4.0 seconds) to prevent infinite loops (like `while True`).
- **Stdout Redirection:** The stdout of the executed script is captured, and the exact deterministic result (e.g., `__EXACT_EXEC_RESULT__=...`) is parsed and returned via the REST API. This ensures the final answer is purely algorithmic, completely bypassing the stochastic and hallucinatory nature of standard text generation.

---

3. Model Size Calculation

To comply strictly with the 8B-class limit (per Q3 in the Official Q&A), our pipeline utilizes a **Dual-LLM architecture**, explicitly separating the generative LLMs from the auxiliary encoder models.

The models in our pipeline are:

1. **Main Generator LLM**: `Qwen2.5-7B-Instruct`
    - **Parameter Count**: **~7.00 B**
    - **Role**: Core reasoning, mathematical solving, and Python code generation engine.
2. **Expansion LLM**: `Gemma-3-1B-it`
    - **Parameter Count**: **~1.00 B**
    - **Role**: High-speed keyword extraction and query expansion prior to hybrid retrieval.

(Note: The pipeline also uses `BAAI/bge-small-en-v1.5` (~0.03B) for embedding and `BAAI/bge-reranker-v2-m3` (~0.56B) for cross-encoding. While these are BERT-based encoders and not generative LLMs, we include them for total transparency).

Total Active Parameter Calculation:

- **Main LLM (7.00B)** + **Expansion LLM (1.00B)** + **Reranker (0.56B)** + **Embedding (0.03B)**.
- While the total combined size across all servers is ~8.59B, our system executes these models **sequentially** (one at a time) per the allowed rules:
    1. Expansion LLM processes the query (**~1.00B** running).
    2. Embedding & Reranker retrieve context (**~0.59B** running).
    3. Main LLM generates the final code (**~7.00B** running).
- Because they are used one at a time, the maximum running parameter count at any single moment is the largest single component, which is **~7.00B**.

Conclusion: The maximum running parameter size at any single moment is **~7.00B**, which strictly complies with the **8B-class limit** constraint. No Mixture of Experts (MoE) models are used.